

Theorem 6.14: Let G be constructed from PDA P . Then $L(G) = N(P)$

$L(G)$ = taal geaccepteerd door CFG G

$N(P)$ = taal geaccepteerd door PDA P , via empty stack (waarom empty stack?)

Bewijs

($\geq$ -direction): We bewijzen door inductie op n : aantal stappen in $\vdash^*$ dat
als $(q, w, X) \vdash^n (p, \epsilon, \epsilon)$, dan $[qXp] \Rightarrow^* w$ ♥

$(q, w, X) \vdash^n (p, \epsilon, \epsilon)$: beginnen in state q , met input string w en stack met symbool X .
PDA maakt exact n transities ($\vdash^n$) om staat p te bereiken
met geen resterende input (ϵ) en een empty stack (ϵ).
PDA accept string w door stack te legen

$[qXp] \Rightarrow^* w$: Variabele $[qXp]$ (dus nonterminal met states p en q met stack symbool X)
leidt string w af door 0 of meerdere productiestappen

basis: Length 1. Then w is an a or ϵ , and $(p, \epsilon) \in \delta(q, w, X)$.
By construction of G , we have $[qXp] \rightarrow w$, and thus $[qXp] \Rightarrow^* w$

Computatie in PDA is 1 move: $(q, w, X) \vdash (p, \epsilon, \epsilon)$. Dit betekent dat $(p, \epsilon) \in \delta(q, a, X)$
waar $a = w$ als w een terminalsymbool is, of $\epsilon \in \Sigma \cup \{\epsilon\}$

Als we dan G construeren, is er een productie $[qXp] \rightarrow w$. (zie omzetting PDA naar CFG)
Dus ook $[qXp] \Rightarrow w$, dus ook $[qXp] \Rightarrow^* w$.

De basisstap toont de simpelste case waar PDA X popt, zonder iets te pushen, en w consumeert in 1 stap.

inductie: Length is $n > 1$, and ♥ holds for lengths $< n$.

We must have a $\vdash$ sequence: $(q, w, X) \vdash (r_0, x, y_1 y_2 \dots y_k) \vdash \dots \vdash (p, \epsilon, \epsilon)$

where $w = ax$ and $a \in \Sigma \cup \{\epsilon\}$.

It follows that $(r_0, y_1 y_2 \dots y_k) \in \delta(q, a, X)$. Then we have production
 $[qXr_k] \rightarrow a[r_0 y_1 r_1] \dots [r_{k-1} y_k r_k]$, for all $\{r_1, \dots, r_{k-1}\} \subseteq Q$ and $r_k = p$.

We beginnen met computatielengte $n > 1$: $(q, w, X) \vdash^n (p, \epsilon, \epsilon)$ (zie 1)

Omdat $n > 1$, moet de eerste move minstens 1 symbool pushen (anders is het gewoon een pop $\rightarrow$ basisstap)
Dan is de eerste transitie: $(q, w, X) \vdash (r_0, x, y_1 y_2 \dots y_k)$ waar $k \geq 1$, $w = ax$ en $a \in \Sigma \cup \{\epsilon\}$.

$\hookrightarrow$ inputsymbool wordt geconsumeerd in deze move.

Dan volgt $(r_0, y_1 \dots y_k) \in \delta(q, a, X)$. Dit is de PDA transitiefunctie output.

Resterende computatie is dan van lengte $n-1$: $(r_0, x, y_1 y_2 \dots y_k) \vdash^{n-1} (p, \epsilon, \epsilon)$

Waarom deze stap? We willen eerste transitie apart nemen, want dit is een grammar production.
De rest is korter, dus kunnen we IH later toepassen.
 $\hookrightarrow$ processen PDA's namelijk hun input: one transition at a time,
met stack veranderingen

We weten bij conversie van PDA naar CFG (pg 81):

voor elke transitie $(r_0, y_1 \dots y_k) \in \delta(q, a, X)$

en voor elke sequentie van intermediaire states $r_1, \dots, r_{k-1} \in Q$

is er een productie: $[qXr_k] \rightarrow a[r_0y_1r_1][r_1y_2r_2] \dots [r_{k-1}y_kr_k]$ ②

($[qXp]$ vervangen we door $[qXr_k]$ met $r_k = p$)

② Waarom deze stap? Deze productieregels simuleren de transities van de PDA.

Linkerkant: begint in state q met X op de stack en eindigt in $r_k = p$

Rechterkant: consumeert a , en zet nonTerminals op om elke gepushte y_i te behandelen, van state r_{i-1} naar r_i

We may choose r_i to be the state of the PDA in the above $\vdash$ sequence when y_i is popped.

In de PDA-berekeningssequentie verwerkt de machine x en poppt elke y_i uiteindelijk op een specifiek moment. We kiezen r_{i-1} als de toestand waarin y_i de top van de stapel wordt (na verwerking van hogere symbolen), en r_i als de toestand direct na het poppen van y_i (na volledige afhandeling van y_i en eventuele erop gepushte symbolen). Dit komt overeen met punten waar de stapelhoogte daalt per y_i .

Omdat de grammatica producties heeft voor alle mogelijke r_i -reeksen, inclusief deze exacte uit de berekening, kunnen we de passende productie selecteren.

Waarom deze stap? De grammatica genereert producties voor elke denkbare tussentoestand-combinatie, zodat voor elk echt PDA-pad een exact matchende productie bestaat. Dit laat de afleiding perfect aansluiten op het specifieke pad.

Let $w = a w_1 w_2 \dots w_k$ where w_i is consumed while y_i is popped. Then: $(r_{i-1}, w_i, y_i) \vdash^* (r_i, e, e)$

$w = a w_1 w_2 \dots w_k$ waar elke w_i de substring van x is.

De PDA begint in toestand r_{i-1} met y_i op de top van de stack en eindigt door y_i te poppen. Het bereikt toestand r_i met lege stapel voor dat subdeel.

Dus voor elke i : $(r_{i-1}, w_i, y_i) \vdash^* (r_i, e, e)$

dese subberekeningen hebben lengte kleiner dan n , want ze tellen op tot $n-1$ (verwijzen?)

By the IH we get $[r_{i-1} y_i r_i] \Rightarrow^* w_i$ ⑤

De subberekening $(r_{i-1}, w_i, y_i) \vdash^* (r_i, e, e)$ heeft lengte kleiner dan n , voor elke $i=1$ tot k .

Dus volgt uit IH: $[r_{i-1} y_i r_i] \Rightarrow^* w_i$

Waarom? De IH geeft ons de afleidingen voor de nonTerminals, die het oplossen van elke gepushte y_i konen

We then get the following derivation sequence:

$$\begin{aligned}
 [q X r_k] &\Rightarrow a[r_0 y_1 r_1] \cdots [r_{k-1} y_k r_k] \Rightarrow^* \\
 &a w_1 [r_1 y_2 r_2] [r_2 y_3 r_3] \cdots [r_{k-1} y_k r_k] \Rightarrow^* \\
 &a w_1 w_2 [r_2 y_3 r_3] \cdots [r_{k-1} y_k r_k] \Rightarrow^* \\
 &\quad \dots \\
 &a w_k w_k \cdots w_k = w \quad \text{and } r_k = p
 \end{aligned}$$

We beginnen met productie ②: $[q X r_k] \Rightarrow a[r_0 y_1 r_1] \cdots [r_{k-1} y_k r_k]$

Zie ⑤: uit IH volgt: $[r_{i-1} y_i r_i] \Rightarrow^* w_i$

Als we nu dan elk nietterminaal in ② vervangen krijgen we: $\Rightarrow^* a w_1 w_2 \cdots w_k = a x = w$

Dus hebben we: $[q X p] \Rightarrow^* w$ (want $r_k = p$)

Dit zouden we bewijzen in ♥

Waarom deze stap? Dit is de volledige afleiding.
 De eerste grammatica stap toont de initiële PDA push.
 Met de daaropvolgende stappen handelen we de subdelen (via IH)
 Zo wordt de hele string w opgeleverd.